

RobustRAG: An Empirical Study of Corpus Poisoning and Query Noise in Retrieval-Augmented Question Answering

Maryam Zaheer

Department of Computer Science

University of Engineering and Technology (UET), Lahore

2023cs175@student.uet.edu.pk

Abstract—Retrieval-Augmented Generation (RAG) systems answer questions by first retrieving relevant text from an external corpus and then conditioning a language model on that text. This design assumes the corpus can be trusted, an assumption that does not hold once anyone other than the system operator can add documents to it. We study this assumption empirically on a dense retrieval pipeline built from a BGE sentence encoder, a FAISS index, and an extractive reader, evaluated on the SQuAD 1.1 validation set (2,067 passages, 10,570 questions). We inject three types of poisoned documents into the corpus — verbatim duplicates, fact-altered duplicates, and word-shuffled noise — at a 5% injection rate, and separately corrupt query embeddings with Gaussian noise. We then test a common lightweight defense, near-duplicate suppression, and measure how much of the damage it recovers. Query noise produced by far the largest degradation observed in this work, reducing exact-match answer accuracy by 48% relative to baseline. Corpus poisoning at 5% produced smaller effects on retrieval ranking (a 1.2–2.6% relative drop in MRR depending on strategy) with almost no measurable change in final answer accuracy. Most notably, the near-duplicate suppression defense did not recover any of the retrieval loss it targeted; instead it reduced Recall@5 further, by up to 4.6% relative to the already-poisoned condition, while leaving every answer-quality metric completely unchanged. We trace this to a structural property of the suppression algorithm rather than a tuning problem, and argue it generalizes to any defense that filters a candidate list without reconsidering the top-ranked result.

Index Terms—retrieval-augmented generation, corpus poisoning, dense retrieval, adversarial robustness, question answering

I. INTRODUCTION

A RAG pipeline has two places where it can be attacked, and they are not the same attack. An adversary can corrupt the corpus the retriever searches over, which is the scenario most existing work on RAG security studies, or an adversary (or simple system noise) can corrupt the query representation that the retriever uses to search, which is comparatively unstudied outside of general embedding-robustness literature. Our aim in this project was not to design a new attack or a state-of-the-art defense. It was to build a small, fully reproducible pipeline that measures both failure modes side by side, on the same corpus, under the same retriever, so that their relative severity could actually be compared rather than assumed.

We evaluate three ways of poisoning a document corpus that differ in how aggressively they disguise themselves as

legitimate content, one way of corrupting the query side that is independent of corpus content entirely, and one lightweight defense intended to catch the corpus-side attacks. All three poisoning strategies are constructed from the real corpus rather than from arbitrary injected text, so that any observed degradation can be attributed to the poisoning mechanism itself and not to some unrelated distributional shift.

The main contribution of this paper is not a novel attack or a novel defense. It is an evaluation that produced a specific, mechanistically explainable negative result: a standard-looking defense against near-duplicate poisoning, implemented the way most practitioners would implement it on a first attempt, measurably fails at the one thing it is meant to do, and we can point to exactly why. We think this is more useful to report than a defense that happened to work, because the reason it fails is not specific to our implementation. It follows from how the algorithm is structured, and the same structure appears in any greedy top-down suppression method.

II. RELATED WORK

Retrieval-augmented generation was introduced by Lewis et al. [1] as a way to ground language model outputs in an external, updatable knowledge source rather than relying solely on parameters learned at training time. Dense retrieval itself, the approach used here in place of sparse lexical matching, follows the dual-encoder framework popularized for open-domain question answering by Karpukhin et al. [2], and our embedding model is one of the general-purpose sentence encoders released as part of the BGE family [3].

The security implications of trusting a retrieval corpus have received attention only recently. Zhong et al. [7] show that a small number of adversarially optimized passages can be injected into a retrieval corpus to consistently surface for a chosen set of queries, using a gradient-based attack against the retriever’s embedding space. Zou et al. [8] extend this to full RAG pipelines with PoisonedRAG, formalizing corpus poisoning as an optimization problem and demonstrating attack success rates above 90% against black-box RAG systems with as few as five injected passages per target question. Both of these are considerably stronger attacks than what we study here: they optimize the poisoned text specifically to maximize retrieval rank, whereas our poisoning strategies are

heuristic and derived directly from existing corpus content. We see this as a deliberate scope difference rather than a shortcoming to be excused. Our goal was to characterize what happens under attacks a non-adversarial-ML practitioner could plausibly construct and reason about, and to test whether an equally simple defense holds up against them, not to reproduce the strongest attacks in the literature.

III. METHODOLOGY

A. Dataset

We use the SQuAD 1.1 validation split [5], not the training split. This choice matters for a reason specific to our evaluation setup: the extractive reader we use for answer extraction, `distilbert-base-cased-distilled-squad`, was fine-tuned on SQuAD’s training split. Evaluating on that same split would risk the reader answering from memorized question-answer associations rather than from the passage it was actually given, which would confound any attempt to measure how retrieval quality affects answer quality. The validation split avoids this. It contains 48 articles, which we parse into 2,067 unique paragraphs after deduplication, paired with 10,570 answerable questions, each with a known gold passage.

B. Retrieval System

Passages and queries are embedded with `bge-small-en-v1.5` [3], a compact sentence encoder trained for text-to-text retrieval rather than image-text alignment, which was our reason for not using CLIP despite it being a natural first candidate for an embedding model. BGE encodes queries and passages asymmetrically: queries receive an instruction prefix (“Represent this sentence for searching relevant passages:”) that passages do not, which we found makes a measurable difference to retrieval quality and is easy to miss if implementing this pipeline from a generic sentence-embedding tutorial. Embeddings are L2-normalized and indexed with FAISS [4] using exact inner-product search (`IndexFlatIP`), which at a corpus size of roughly two thousand passages is fast enough that approximate indexing offers no practical benefit.

C. Poisoning Strategies

We construct three families of poisoned documents, all derived from the existing corpus and appended to it rather than replacing any original content, at an injection rate of 5% (103 additional documents):

- **Near-duplicate.** A verbatim copy of a randomly selected passage. This represents the simplest possible corpus attack: flooding the index with redundant copies of legitimate content.
- **Contradictory.** A copy of a passage in which the correct answer string is replaced with an incorrect answer string drawn from elsewhere in the corpus, when that string occurs verbatim in the passage text. Where the substring match fails, we fall back to a plain duplicate rather than silently skip the passage. This strategy is intended to be

the most dangerous of the three, since it is embedding-close to genuine content but asserts something false.

- **Irrelevant.** A word-shuffled version of a passage. The resulting text shares vocabulary with a real passage but has no coherent meaning, testing whether retrieval can be misled by superficial lexical similarity alone.

D. Query Noise

Independently of corpus poisoning, we perturb query embeddings with additive Gaussian noise (standard deviation 0.1) before renormalizing, then retrieve as usual. This models a corrupted or imprecise query representation rather than a corrupted corpus, and we kept it structurally separate from the poisoning experiments so that its effect could be measured without confounding it with corpus-side changes.

E. Defense

We implement near-duplicate suppression as our one lightweight defense. Given a candidate pool of 20 retrieved documents (four times the final top- k of 5), the algorithm iterates candidates in descending score order and keeps a candidate only if its embedding similarity to every already-kept candidate falls below a threshold of 0.97 cosine similarity; otherwise it is treated as a duplicate and skipped, with lower-ranked candidates backfilled until 5 remain. The intent was to catch near-duplicate and contradictory poisoning specifically, since both are constructed to sit close to a genuine passage in embedding space, while leaving the irrelevant strategy untouched, since word-shuffled text does not produce a near-duplicate embedding of anything.

F. Answer Extraction and Metrics

For each query, we extract an answer from the top-ranked retrieved passage using `distilbert-base-cased-distilled-squad`, an extractive question-answering model. We deliberately use only the top-1 passage rather than concatenating several, so that answer quality remains directly attributable to top-ranked retrieval quality rather than to the reader’s ability to work around a bad top result using lower-ranked context. We report Recall@5, Precision@5, and Mean Reciprocal Rank for retrieval, and Exact Match, BLEU, and ROUGE-L for answer quality, computed against SQuAD’s human-annotated gold answers.

IV. EXPERIMENTAL SETUP

All experiments run on CPU (a Kaggle notebook instance), with a fixed random seed of 42 governing corpus subsampling, poisoning construction, and noise generation. Retrieval metrics are computed over the full set of 10,570 evaluable queries. We observed minor non-determinism across repeated runs of the same configuration (differences in the fourth decimal place of Recall@5 for the contradictory condition across two otherwise identical runs), most likely attributable to floating-point summation order under multi-threaded CPU execution in FAISS or PyTorch. This is small enough not to affect any

conclusion drawn here, but we note it explicitly rather than presenting our numbers as more exactly reproducible than they are.

V. RESULTS

Table I reports all measured conditions relative to the clean baseline.

A. Query noise dominates every other effect

The gap between the clean baseline and the noisy-query condition is an order of magnitude larger than anything produced by corpus poisoning at 5%: Recall@5 falls by 32% relative, MRR by 42%, and Exact Match by 48%. Every metric moves in the same direction and by a comparable relative amount, which is consistent with noise acting on the query representation directly rather than selectively corrupting some particular part of the pipeline.

B. Poisoning at 5% is a modest but mechanistically consistent effect

Corpus poisoning at a 5% injection rate produces changes in the low single digits of relative degradation for retrieval ranking, and close to no change for final answer accuracy in the near-duplicate case specifically. This is not a null result so much as a structural one: a near-duplicate poisoned document is textually identical to the passage it copies, so on the queries where it wins a ranking slot, the text handed to the reader is unchanged regardless of which copy the retriever selects. Exact Match, BLEU, and ROUGE-L for the near-duplicate condition are numerically identical to the baseline to sixteen significant digits, which is not a coincidence, it is what the mechanism predicts. The contradictory strategy, which does alter document content, is the only poisoning condition that produces any measurable movement in answer-quality metrics at all, and MRR degradation ordering across the three strategies (near-duplicate worst, then contradictory, then irrelevant) matches the expectation that embedding-close attacks disrupt ranking more than an attack that merely shares vocabulary.

C. The defense does not recover retrieval loss, and cannot affect answer quality by construction

This is the central finding of the paper. Near-duplicate suppression, applied on top of the near-duplicate poisoning condition it was designed for, reduces Recall@5 further, from 0.884 down to 0.843, a 4.6% relative drop measured against the already-poisoned baseline, not against clean. The contradictory condition shows the same direction of effect at smaller magnitude (0.883 to 0.873, a 1.2% relative drop). The irrelevant condition is unaffected in every metric, exactly as expected, since word-shuffled documents do not produce near-duplicate embeddings for the suppression threshold to catch.

Across all three defended conditions, Exact Match, BLEU, and ROUGE-L are identical to their corresponding undefended poisoned values. This is not because the defense has no effect on retrieval; the previous paragraph shows that it clearly does.

It is because of how the suppression algorithm is structured. It builds its kept-candidate list greedily in descending score order, and the first candidate is always kept unconditionally, since there is nothing yet in the kept list to compare it against. The top-ranked result after defense is therefore always identical to the top-ranked result before defense, for every single query, regardless of what that top result is. Since the reader only ever sees the top-ranked passage, the defense is structurally incapable of changing what the reader is given, and therefore incapable of changing any answer-quality metric, no matter how well or badly it is tuned. What it can and does change is the composition of positions two through five, and in our results that change was net negative for retrieval.

VI. DISCUSSION

The negative result in Section V-C is worth taking seriously beyond this specific implementation. Any defense that filters a fixed-size candidate list top-down without reconsidering whether the top result itself should be replaced will share this property: it can improve or damage recall and precision within the tail of the ranking, but it cannot touch the position that determines what a downstream reader actually sees, unless the top result happens to be removed for some other reason. A defense meant to protect answer quality specifically needs to be willing to demote the top-ranked candidate, not just filter around it. That this needs to be designed for explicitly, rather than falling out naturally from a reasonable-looking similarity filter, was the most useful thing this project surfaced, more useful in our view than if the defense had simply worked.

The retrieval-side regression is a separate and smaller issue, but also worth explaining rather than leaving as an unexplained number. With a candidate pool of 20 and a poisoning rate of 5%, most individual queries will not have more than one or two poisoned candidates in their pool, so most of the suppression activity is likely operating on pairs of genuinely similar clean documents, for instance two paragraphs from the same article discussing closely related content, rather than on the poisoned documents specifically. A threshold tuned without checking its false-positive rate on unpoisoned near-duplicate-like content will do exactly this. We did not tune the threshold against that criterion in this work, which we consider the most direct next step rather than a flaw to excuse.

VII. LIMITATIONS

- All poisoning experiments were run at a single injection rate (5%). We do not know whether the defense’s negative effect on Recall@5 scales, shrinks, or reverses at higher or lower rates, and a dose-response sweep across rates would be needed to say anything about how these results generalize.
- We report point estimates without a paired significance test. The 5% poisoning conditions in particular produce effects small enough (well under 1% relative for most metrics) that we cannot currently rule out that some portion of the observed difference is within the noise floor of a single seeded run rather than a real effect

TABLE I
RETRIEVAL AND ANSWER-QUALITY METRICS UNDER EACH CONDITION. ALL POISONING CONDITIONS USE A 5% INJECTION RATE.

Condition	Recall@5	Precision@5	MRR	EM	BLEU	ROUGE-L
Clean (baseline)	0.887	0.177	0.763	0.559	0.259	0.631
Near-duplicate, poisoned	0.884	0.177	0.744	0.559	0.259	0.631
Near-duplicate, defended	0.843	0.169	0.726	0.559	0.259	0.631
Contradictory, poisoned	0.883	0.177	0.750	0.558	0.258	0.629
Contradictory, defended	0.873	0.175	0.750	0.558	0.258	0.629
Irrelevant, poisoned	0.886	0.177	0.761	0.557	0.258	0.628
Irrelevant, defended	0.886	0.177	0.761	0.557	0.258	0.628
Query noise ($\sigma = 0.1$)	0.603	0.121	0.443	0.290	0.136	0.342

of poisoning. A McNemar’s test on paired per-query retrieval hit/miss outcomes would address this directly, but requires per-query outcome logging that the current pipeline does not retain, only aggregate metrics.

- The poisoning strategies studied here are heuristic and derived from real corpus content, not adversarially optimized against the retriever. Stronger, gradient-based attacks such as those in [7] or [8] would likely produce substantially larger effects than what we report, and we would not expect our defense’s failure mode to change under those attacks, since the structural argument in Section V-C does not depend on how the poisoned document was constructed.
- The extractive reader constrains the answer format to a contiguous span of the retrieved passage. It cannot correct, synthesize, or hedge, which likely understates how badly a generative reader’s answer could be corrupted by the same retrieval failures.
- Recall@k, Precision@k, and MRR as computed here assume exactly one relevant passage per query, which holds for this SQuAD-derived setup but would need to be generalized for corpora with multiple relevant passages per query.
- We observed small non-deterministic variation across repeated runs of the same seeded configuration, discussed in Section IV, which we did not fully diagnose or eliminate.

VIII. FUTURE WORK

The most direct extension is redesigning the defense to allow the top-ranked candidate to be reconsidered rather than protected by construction, for instance by comparing every candidate, including the top one, against a reference fingerprint of the unpoisoned corpus rather than only against other retrieved candidates. A second direction is a poisoning-rate sweep (for example 5%, 10%, 20%) to establish whether the defense’s negative retrieval effect scales with attack intensity or is closer to a fixed cost independent of how much poisoning is present. Adding per-query outcome logging to support a proper paired significance test would let the smaller poisoning effects reported here be stated with actual statistical confidence rather than as observed differences. Finally, comparing this lightweight defense against a cross-encoder re-ranking

baseline would establish whether the recall regression we observed is a general property of post-hoc filtering approaches or specific to similarity-threshold suppression.

IX. CONCLUSION

We built and evaluated a RAG robustness pipeline on a real dense retriever and a real 10,570-question benchmark, and found that query noise, not corpus poisoning, was the dominant threat to answer quality at the settings we tested. More importantly, we found that a defense designed to counter corpus poisoning not only failed to recover the loss it targeted, it made retrieval measurably worse, and we can explain exactly why: the algorithm’s greedy top-down structure makes it structurally incapable of changing the one ranking position that determines what the downstream reader sees. We think this is a more useful outcome to report than a defense that appeared to work, because the explanation generalizes beyond this specific implementation to an entire class of post-hoc filtering defenses.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.
- [2] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, 2020.
- [3] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: Packaged resources to advance general Chinese embedding,” *arXiv:2309.07597*, 2023.
- [4] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, 2019.
- [5] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text,” in *Proc. EMNLP*, 2016.
- [6] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv:1910.01108*, 2019.
- [7] Z. Zhong, Z. Huang, A. Wettig, and D. Chen, “Poisoning retrieval corpora by injecting adversarial passages,” in *Proc. EMNLP*, 2023, pp. 13764–13775.
- [8] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” *arXiv:2402.07867*, 2024.